

GagneJèl - Documentation Technique Complète

Plateforme de Paris Sportifs Décentralisée

Vue d'ensemble du projet

GagneJèl est une plateforme de paris sportifs décentralisée construite sur la blockchain Base (Layer 2 d'Ethereum). Elle permet aux utilisateurs de parier entre amis en toute transparence avec des smart contracts sécurisés.

Fonctionnalités clés

- Dépôts et retraits en USDC via smart contracts
- Système de paris multiples (DUO, TRIO, QUATRO, 5 participants)
- Groupes illimités pour parier entre amis
- Résolution automatique des paris
- Distribution automatique des gains
- Interface Web3 avec connexion wallet
- Frais de plateforme de 5%

Architecture Globale

Architecture Technique

Le projet suit une architecture Web3 moderne en 3 couches :

- **Blockchain (Smart Contracts):** Gestion sécurisée des fonds et logique on-chain
- **Backend API (Node.js):** Logique métier et gestion des données off-chain
- **Frontend (Next.js):** Interface utilisateur et interaction Web3

Smart Contract

Composant	Technologie
Langage	Solidity 0.8.20
Framework	Hardhat 2.19.5
Librairies	OpenZeppelin Contracts 5.4.0
Réseau Testnet	Base Sepolia (Chain ID: 84532)
Réseau Mainnet	Base (Chain ID: 8453)
Contract Address	0xeD764c14F9B0c2BED250bD3d00AA117128294ABC
USDC Address	0x036CbD53842c5426634e7929541eC2318f3dCF7e

Fonctionnalités du Smart Contract

- deposit(uint256 amount) - Déposer de l'USDC
- withdraw(uint256 amount) - Retirer de l'USDC
- transferBetween(address from, address to, uint256 amount) - Transférer entre utilisateurs
- batchTransfer(...) - Distribution des gains en batch
- getBalance(address user) - Consulter la balance
- Events (Deposit, Withdrawal, Transfer, BatchTransferCompleted)

Sécurité

- ReentrancyGuard - Protection contre les attaques de réentrance
- Pausable - Système de pause d'urgence
- Ownable - Gestion des permissions administrateur
- Montants min/max configurables
- Events pour traçabilité complète

Stack Backend

Technologies Backend

Composant	Technologie
Runtime	Node.js 18+
Framework	Express.js 4.x
Langage	JavaScript (ES6+)
Base de données	PostgreSQL (via Supabase)
ORM	Prisma 5.x
Blockchain SDK	ethers.js 6.x
Authentication	JWT (JSON Web Tokens)
WebSocket	Socket.io (pour chat temps réel)

Structure Backend

```
gagnejel-backend/
├── src/
│   ├── config/           # Configuration (DB, Blockchain)
│   ├── models/           # Modèles Prisma
│   ├── routes/           # Routes API REST
│   ├── controllers/      # Logique métier
│   ├── services/         # Services (blockchain, etc)
│   ├── middleware/       # Auth, validation
│   ├── utils/            # Fonctions utilitaires
│   └── server.js         # Point d'entrée
├── prisma/
│   └── schema.prisma     # Schéma base de données
├── .env
└── package.json
```

API Routes

Endpoint	Description
GET /api/health	Health check
GET /api/users	Liste des utilisateurs
POST /api/users	Créer un utilisateur
GET /api/matches	Liste des matchs
GET /api/matches/upcoming	Matchs à venir
POST /api/matches	Créer un match (admin)
PATCH /api/matches/:id/resolve	Résoudre un match (admin)
GET /api/bets	Liste des paris
POST /api/bets	Placer un pari
GET /api/bets/user/:userId	Paris d'un utilisateur
POST /api/bets/match/:matchId/resolve	Résoudre les paris d'un match
GET /api/blockchain/contract/info	Infos du smart contract
GET /api/blockchain/balance/:address	Balance blockchain d'un wallet

Modèles de Données (Prisma)

- **User:** Utilisateurs de la plateforme (walletAddress, username, stats)
- **Match:** Matches sportifs (homeTeam, awayTeam, score, winner)
- **Bet:** Paris placés (type, prediction, amount, odds, status)
- **Group:** Groupes d'amis (name, members, stats)
- **GroupMember:** Membres des groupes (userId, groupId, role)
- **Message:** Messages de chat (content, userId, groupId)

Service Blockchain

- Initialisation de la connexion au smart contract
- Récupération des balances utilisateurs
- Transferts entre utilisateurs (backend only)
- Distribution batch des gains
- Écoute des events blockchain
- Gestion des erreurs et retry logic

Technologies Frontend

Composant	Technologie
Framework	Next.js 14 (App Router)
Langage	TypeScript 5.x
Styling	Tailwind CSS 3.x
Web3 Library	Wagmi 2.x
Ethereum Library	Viem 2.x
Wallet Connection	RainbowKit 2.x
State Management	React Query (TanStack Query)
HTTP Client	Axios
Icons	Lucide React
Notifications	React Hot Toast

Structure Frontend

```
gagnejel-frontend/
├── app/
│   ├── page.tsx           # Page d'accueil
│   ├── dashboard/
│   │   └── page.tsx       # Dashboard utilisateur
│   ├── matches/
│   │   └── [id]/page.tsx  # Détail match
│   ├── layout.tsx         # Layout principal
│   ├── globals.css        # Styles globaux
│   └── providers.tsx       # Web3 Providers
├── lib/
│   ├── wagmi.ts           # Config Wagmi
│   └── api/
│       └── client.ts       # API client
├── components/            # Composants réutilisables
├── .env.local
└── package.json
```

Fonctionnalités Frontend

- Connexion wallet (MetaMask, Coinbase Wallet, WalletConnect)
- Dashboard utilisateur avec stats
- Liste des matchs disponibles
- Placement de paris avec sélection du type
- Historique des paris
- Profil utilisateur
- Notifications temps réel
- Responsive design (mobile-first)

Infrastructure & Déploiement

Environnements

Environnement	Configuration
Development	localhost:3000 (frontend), localhost:5000 (backend)
Testnet	Base Sepolia pour tests
Production	Base Mainnet (à déployer)

Services Externes

- **Supabase:** PostgreSQL hébergé (gratuit jusqu'à 500MB)
- **Base RPC:** <https://sepolia.base.org> (testnet), <https://mainnet.base.org> (prod)
- **Basescan:** Explorateur blockchain pour vérification
- **Coinbase Faucet:** ETH testnet gratuit
- **GitHub:** Hébergement du code source

Variables d'Environnement

Backend (.env):

```
PORT=5000
NODE_ENV=development
DATABASE_URL=postgresql://...
JWT_SECRET=...
ESCROW_CONTRACT_ADDRESS=0xeD764c14...
PRIVATE_KEY=0x...
BASE_SEPOLIA_RPC=https://sepolia.base.org
CHAIN_ID=84532
USDC_ADDRESS=0x036CbD53...
```

Frontend (.env.local):

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
NEXT_PUBLIC_ESCROW_CONTRACT_ADDRESS=0xeD764c14...
NEXT_PUBLIC_USDC_ADDRESS=0x036CbD53...
NEXT_PUBLIC_CHAIN_ID=84532
```

Workflow de Développement

Setup du Projet

1. Smart Contract:

```
cd gagnejel-blockchain
npm install
npx hardhat compile
npx hardhat run scripts/deploy.js --network baseSepolia
```

2. Backend:

```
cd gagnejel-backend
npm install
cp .env.example .env
# Éditer .env avec vos valeurs
npx prisma migrate dev --name init
npm run dev
```

3. Frontend:

```
cd gagnejel-frontend
npm install
cp .env.example .env.local
# Éditer .env.local avec vos valeurs
npm run dev
```

Commandes Utiles

Composant	Commande	Description
Backend	npm run dev	Démarrer le serveur de développement
Frontend	npm run dev	Démarrer Next.js dev server
Smart Contract	npx hardhat compile	Compiler les contracts
Smart Contract	npx hardhat test	Lancer les tests
Backend	npx prisma studio	Interface graphique DB
Backend	npx prisma migrate dev	Créer une migration

Sécurité

Smart Contract

- Utilisation d'OpenZeppelin (bibliothèques auditées)
- ReentrancyGuard sur toutes les fonctions critiques
- Système de pause d'urgence (Pausable)
- Contrôle d'accès (Ownable)
- Validation des montants (min/max)
- Events pour traçabilité
- Tests unitaires et d'intégration

Backend

- JWT pour l'authentification
- Validation des inputs (express-validator)
- CORS configuré
- Variables d'environnement sécurisées
- Rate limiting (à implémenter)
- Clés privées chiffrées
- Logs sécurisés (pas de données sensibles)

Frontend

- Pas de clés privées stockées côté client
- Signature des transactions via wallet
- Validation côté client ET serveur
- HTTPS en production
- Protection contre XSS (Next.js par défaut)
- Content Security Policy
- Pas de données sensibles dans le localStorage

Performance & Optimisation

Blockchain

- Utilisation de Base L2 (frais 10-100x moins chers qu'Ethereum)
- Batch transfers pour économiser du gas
- Events plutôt que storage pour historique
- Optimisation du code Solidity (runs: 200)

Backend

- Prisma pour requêtes optimisées
- Indexes sur colonnes fréquemment requêtées
- Pagination sur toutes les listes
- Cache Redis (à implémenter)
- Connection pooling PostgreSQL

Frontend

- Next.js 14 avec App Router (performance optimale)
- Server Components quand possible
- Images optimisées (next/image)
- Code splitting automatique
- React Query pour cache des données
- Lazy loading des composants
- Tailwind CSS (CSS minimal en production)

Tests & Qualité

Smart Contract

Tests avec Hardhat + Chai:

- Tests unitaires (fonctions individuelles)
- Tests d'intégration (flux complets)
- Tests de sécurité (attaques potentielles)
- Tests de gas (optimisation)
- Tests sur réseau local (Hardhat Network)

Backend

Tests avec Jest + Supertest:

- Tests unitaires (controllers, services)
- Tests d'intégration (API endpoints)
- Tests de base de données (Prisma)
- Tests blockchain (mocks)

Frontend

Tests avec Jest + React Testing Library:

- Tests unitaires (composants)
- Tests d'intégration (pages)
- Tests E2E avec Playwright (recommandé)

Roadmap & Évolutions Futures

Phase 1 - MVP (Complété à 75%)

-  Smart contract déployé sur testnet
-  Backend API avec CRUD complet
-  Connexion wallet frontend
-  Placement de paris basique
-  Dashboard utilisateur
-  Page détail match
-  Historique paris

Phase 2 - Améliorations (2-3 semaines)

- Groupes d'amis fonctionnels
- Chat temps réel (Socket.io)
- Notifications push
- Profils utilisateurs avancés
- Système de classement
- API externe pour résultats matchs
- Tests automatisés complets

Phase 3 - Production (1 mois)

- Déploiement smart contract sur Base Mainnet
- Audit de sécurité du smart contract
- Infrastructure de production (Vercel + Railway)
- Monitoring et alertes (Sentry)
- Analytics (Mixpanel/PostHog)
- SEO et marketing
- Programme de beta testing

Phase 4 - Scale (3-6 mois)

- Application mobile (React Native)
- Intégration d'autres sports
- Système de récompenses/tokens
- Partenariats avec ligues sportives
- Support multi-devises
- Expansion internationale

Ressources & Documentation

Documentation Officielle

- **Solidity:** <https://docs.soliditylang.org>
- **Hardhat:** <https://hardhat.org/docs>
- **OpenZeppelin:** <https://docs.openzeppelin.com>
- **Base:** <https://docs.base.org>
- **Next.js:** <https://nextjs.org/docs>
- **Wagmi:** <https://wagmi.sh>
- **RainbowKit:** <https://www.rainbowkit.com>
- **Prisma:** <https://www.prisma.io/docs>
- **Express:** <https://expressjs.com>

Outils Utiles

- **Basescan (Testnet):** <https://sepolia.basescan.org>
- **Coinbase Faucet:** <https://portal.cdp.coinbase.com/products/faucet>
- **Supabase:** <https://supabase.com>
- **Vercel:** <https://vercel.com>
- **GitHub:** <https://github.com/omzo761/gagnejel>

Équipe & Contact

Développé par une équipe de passionnés de blockchain et de sports.

 Contact: contact@gagnejel.com

 Twitter: [@gagnejel](https://twitter.com/gagnejel)

 Discord: discord.gg/gagnejel